Architectural Optimization and Governance Refactoring Report: Operational De-Overloading and The Quartet Filing Doctrine

1. Executive Summary

This comprehensive research report presents a rigorous architectural analysis of the *Stela* repository state (specifically targeting HEAD 80bb3afd), responding to the Operator's mandate to identify and remediate functional overloading within the ops/ directory. The primary objective is to engineer a code reduction of 12% or greater while simultaneously integrating the "Quartet Filing Doctrine" introduced in DP-OPS-0068 and strictly adhering to the "Policy of Truth" (PoT) governance model.

The analysis indicates that the repository, while robust in its constitutional definitions, suffers from significant implementation drift in its operational tooling. The current tooling suite exhibits a pattern of "monolithic accretion," where binaries such as ops/bin/project, ops/bin/template, and ops/bin/dump have accumulated disparate responsibilities—conflating instantiation with verification, and expansion with synthesis. This architectural anti-pattern violates the "Separation of Concerns" principle, increases the "blast radius" of routine maintenance, and directly contributes to code bloat through the duplication of path-handling logic and conditional routing. Furthermore, the existing storage topology relies on an overloaded storage/ hierarchy that intermingles transient runtime data, historical archives, and active handoff artifacts, violating the constitutional "Physical Laws" of isolation defined in PoT.md.

The proposed remediation strategy centers on a "Binary Fission" methodology, decoupling overloaded tools into specialized, single-purpose executables (e.g., separating project-scaffold from project-meta), combined with the adoption of the Quartet Filing Doctrine. This doctrine re-architects the repository root into four distinct domains: var/ (Resume), logs/ (Telemetry), archives/ (Cold), and storage/ (Payload). By implementing a centralized "Registry Pattern" for path governance, the system can eliminate redundant regex-based policing logic across CI/CD workflows, achieving the targeted code reduction. This report details the theoretical justification, mechanical implementation steps, and strategic options for executing this transformation, ensuring a streamlined, deterministic, and constitutionally aligned operational environment.

2. Analysis: Governance Architecture and Repository

State

To effectively prescribe a refactoring strategy, one must first deconstruct the current governance architecture and the physical state of the repository. The *Stela* system operates not merely as a collection of code but as a jurisdiction governed by text-based law. Any structural change must be validated against these constitutional constraints.

2.1 The Constitutional Framework (PoT and Axioms)

The "Policy of Truth" (PoT.md) serves as the supreme authority within the repository, establishing the "Physical Laws" that dictate file placement and system behavior.[1] A critical analysis of the PoT reveals specific axioms that are currently under strain due to the identified overloading.

- **The Filing Doctrine Axiom:** The PoT explicitly defines the roles of top-level directories: ops/ is for "Run" (binaries, automation), docs/ is for "Explain," opt/ is for "Isolate," projects/ is for "Work," and storage/ is for "Trash" (local artifacts, never canon).[1] The current implementation of ops/bin/prune and the harvester scripts violates the spirit of this doctrine by treating storage/ not just as trash, but as a repository for valuable history (archives) and active state (handoffs). This conflation risks data loss, as "Trash" implies dispensability, yet archives are critical for provenance.
- **The Drift Axiom:** The PoT states that "any divergence between canon and repository state... is a failure state that requires stop and correction".[1] Overloaded binaries increase the probability of drift. When a single binary like ops/bin/project manages both scaffolding (files) and governance (registry checks), a bug in the file creation logic can block the governance checks, effectively disabling the system's ability to detect drift. This coupling endangers the "Integrator" role's ability to maintain structural integrity.
- **The Single Source of Truth (SSOT) Axiom:** The PoT mandates "one canonical file per domain".[1] However, the analysis of the dump reveals that path definitions (e.g., what constitutes a "code-bearing surface") are duplicated across multiple policing workflows (pow_policing.yml, sop_policing.yml) and scripts.[1] This duplication constitutes a violation of the SSOT axiom, as a change to the definition of "code" requires synchronized updates across multiple YAML and Bash files.

2.2 The Pathology of Overloading in ops/

The "2 1 1 Overloading in ops.md" research material identifies a systemic issue where operational tools have expanded beyond their original scope.[1] This "mission creep" is the primary driver of excess code lines and brittleness.

2.2.1 The Instantiation/Governance Conflation (ops/bin/project)

The ops/bin/project binary is identified as a critical nexus of overloading. It currently encapsulates two fundamentally opposing lifecycles:

1. **Instantiation (The Factory):** This logic is responsible for creating something from nothing. It involves mkdir, touch, and cp operations based on the ops/lib/project/SCAFFOLD.md template.[1] It creates the physical reality of a project.
2. **Governance (The Auditor):** This logic is responsible for verifying that an existing reality matches the law. It checks docs/ops/registry/PROJECTS.md for valid IDs and validates STELA.md compliance.[1]

Architectural Analysis: Coupling these creates a "Privilege Paradox." In a secure pipeline, the "Auditor" (verification) should be able to run with read-only permissions to verify state. However, because it is bound to the "Factory" (instantiation), the binary requires write permissions to function, violating the principle of least privilege. Furthermore, the logic required to parse arguments for init (creation) versus check (verification) introduces unnecessary conditional complexity (case statements), contributing to the code line count.

2.2.2 The Expansion/Synthesis Conflation (ops/bin/template)

The ops/bin/template binary is tasked with two distinct forms of text processing that require different safety guarantees.[1]

1. **Definition Expansion:** This is a "one-to-many" generative process. It takes a template from opt/_factory (e.g., an Agent definition) and instantiates a new file, often requiring unique ID generation and registry lookups. This is a state-mutating operation.
2. **Manifest Synthesis:** This is a "many-to-one" compilation process. It reads ops/src/manifests and aggregates them into ops/lib/manifests/CONTEXT.md. This requires deterministic hazard detection (ensuring opt/ files do not leak into the global context).[1]

Architectural Analysis: Mixing mutable generation with immutable compilation creates non-determinism risks. If the template binary includes logic to "auto-register" a new agent during generation, using that same binary for manifest synthesis could accidentally trigger side effects during a read-only context refresh.

2.2.3 The Hygiene/Policy Conflation (tools/verify.sh)

The tools/verify.sh script currently enforces both structural existence and content policy.[1]

1. **Structural Hygiene:** Checking if ops/, docs/, and tools/ exist. These checks are instant (test -d).
2. **Content Policy:** Scanning every file in docs/ to ensure it is not a binary, and every file in ops/ to ensure it is not a stray markdown file. These checks are expensive (I/O intensive).

Architectural Analysis: This conflation prevents the use of verification in high-frequency loops (e.g., git hooks). Developers bypass the slow policy checks, leading to drift. Splitting these allows "Fast Verification" (tree structure) to run on every commit, while "Deep Verification" (policy) runs in CI.

2.3 Redundancy in Policing Workflows

A deep inspection of the .github/workflows directory reveals massive logic duplication, a prime target for the 12% code reduction goal.[1]

- **pow_policing.yml:** Iterates through git diff, filters for ops/|tools/|projects/, and checks PoW.md.
- **sop_policing.yml:** Iterates through git diff, filters for PoT.md|TASK.md|docs/, and checks SoP.md.
- **task_policing.yml:** Iterates through git diff, filters for "gate surfaces" (ops/bin/open, etc.), and checks TASK.md.

Architectural Analysis: Each workflow re-implements the "detect change -> filter path -> check companion" logic pattern. The definition of what constitutes a "Code Surface" or a "Canon Surface" is hardcoded into the YAML regex. This is brittle; adding a new code directory requires editing multiple YAML files. This violates the "Reuse-first" axiom of the PoT.

2.4 The prune Anomaly

The user query specifically highlights prune as a target for optimization.[1] Currently, ops/bin/prune is overloaded with:

1. **Retention:** Moving old ledger entries to archives.
2. **Hygiene:** Scrubbing storage/tmp.
3. **Restoration:** Resetting TASK.md from a template.

Architectural Analysis: The "Restoration" logic (reset-task) is a severe violation of the Single Responsibility Principle. A tool named prune (removal) should never be responsible for *creating* or *restoring* critical files. This logic belongs in ops/bin/draft or ops/bin/template. This overload forces prune to carry dependencies on the template rendering engine and linting tools, bloating its footprint.

3. The Quartet Filing Doctrine (DP-OPS-0068) Integration

The "Quartet Filing Doctrine," as defined in DP-OPS-0068 [1], represents a fundamental shift in the repository's topology. It replaces the overloaded storage/ directory with four distinct, semantic root directories. This topological shift is the enabler for the proposed code reduction.

3.1 The New Topological Taxonomy

The doctrine establishes four new roots to replace the legacy storage/ model:

New Root	Role	Legacy Equivalent	Constitutional Behavior (PoT)
var/	**Resume**	storage/tmp/	**Ephemeral.** Machine-local runtime state (sockets, PIDs, buffers). Safe to scrub at any time. No archival value.
logs/	**Telemetry**	N/A	**Diagnostic.** Structured execution logs and audit trails. distinct from "Ledgers" (SoP/PoW) which are governance proofs.
archives/	**Cold**	storage/archives/	**Immutable.** Historical records moved out of active ledgers. Once written, they are never modified, only read for forensics.
storage/	**Payload**	storage/	**Active.** Strictly for artifact handoff (handoff/) and state dumps (dumps/). These are the input/output ports of the system.

3.2 Impact Analysis on Existing Tooling

Integrating this doctrine requires specific refactoring across the ops/ suite.

- **ops/bin/prune Refactor:**
  - *Current State:* Hardcoded to archive to storage/archives/ and scrub storage/tmp/.
  - *Required State:* Must target archives/root/ for ledger overflow. Must target var/tmp/ for scrubbing. The logic must be updated to respect the immutability of archives/.
- **tools/verify.sh Refactor:**
  - *Current State:* Checks for the existence of storage/tmp and storage/archives.
  - *Required State:* Must enforce the existence of .gitkeep files in var/, logs/, and archives/. It must explicitly *fail* if storage/tmp is re-created, enforcing the migration.
- **Lifecycle Scripts (agent.sh, task.sh, skill.sh):**
  - *Current State:* Harvest drafts to storage/archives/{type}/.
  - *Required State:* Must harvest to archives/{type}/. This ensures that drafts are treated as "Cold" storage until promoted, aligning with the filing doctrine.
- **ops/bin/context & ops/bin/map:**
  - *Current State:* Use storage/tmp for intermediate processing.
  - *Required State:* Must use var/tmp. This isolates processing noise from the storage/ payload directory, ensuring that storage/ only contains valid artifacts for the Operator.

3.3 Convergence with Code Reduction

The adoption of the Quartet Doctrine facilitates code reduction by simplifying path logic. Currently, tools contain conditional checks to distinguish between "tmp" files in storage/ and "real" files in storage/. By separating them into var/ and storage/, simple glob patterns (e.g., rm -rf var/tmp/*) replace complex find commands with exclusion filters.

4. Methodology: The 12% Reduction Calculus

The target of reducing code lines by 12% is ambitious. It cannot be achieved by minor edits; it requires structural deletion. The following methodology outlines how this figure is reached.

4.1 Mechanism A: Deletion via Specialization (Binary Fission)

Monolithic binaries like ops/bin/project contain substantial boilerplate for argument parsing (getopts, case statements) to handle their multiple modes.

- **Analysis:** A typical bash script wrapper with argument parsing consumes ~30-50 lines. By splitting project into project-scaffold and project-meta, we remove the complex routing logic that handles conflicting flags between the two modes.
- **Reduction Source:** Elimination of conditional blocks (e.g., if mode == init then... else if mode == check then...). Specialized scripts are linear: "Do X, then Exit."
- **Estimated Reduction:** ~150 lines across the ops/bin suite.

4.2 Mechanism B: The Registry Pattern (Eliminating Ghost Canon)

The most significant source of duplication is the hardcoded path definitions in the CI workflows (.github/workflows/*.yml) and scripts.[1]

- **The Solution:** Implement a centralized registry at docs/ops/registry/PATHS.md and a single helper script (ops/lib/logic/paths.sh) to parse it.
- **Transformation:**
  - *Current:* grep -Eq '^(ops/|tools/|projects/|\.github/workflows/)' (Repeated 3+ times in YAML).
  - *Future:* tools/detect-surface.sh --type=code-bearing (Called 3 times).
- **Reduction Source:** This removes the need for complex, multi-line regex definitions in YAML. It also allows the deletion of redundant logic in tools/verify.sh that checks for directory existence, as the registry becomes the single source of truth for repository topology.
- **Estimated Reduction:** ~200 lines (mostly in YAML and validation scripts).

4.3 Mechanism C: Logic Consolidation (Lifecycle Scripts)

The harvester scripts (agent.sh, task.sh, skill.sh) share approximately 80% of their logic.[1]

- **Analysis:** Each script independently implements: provenance generation, file existence checks, log appending, and timestamping.
- **The Solution:** Abstract this into ops/lib/scripts/lifecycle.sh.
  - lifecycle_harvest()
  - lifecycle_promote()
- **Reduction Source:** The individual scripts become thin configuration wrappers (defining directories and templates) around the shared library.
- **Estimated Reduction:** ~300 lines.

4.4 Mechanism D: Scope Reduction (Prune)

By strictly limiting prune to deletion logic and removing the reset-task functionality (delegating it to draft), we remove entire blocks of dependency management code.

- **Estimated Reduction:** ~50 lines.

5. Detailed Component Analysis and Refactoring Logic

This section provides the specific analytical reasoning for the refactoring of each targeted component, synthesizing the findings from the dump and the overloading report.

5.1 ops/bin/project: The Decoupling of Creation and Law

Current State: The binary mixes the mechanical act of file creation with the juridical act of validation. This means the "Factory" (creation) implies the "Auditor" (validation).

Refactoring Logic:

- **ops/bin/project-scaffold:** This becomes a "dumb" tool. It takes a name and creates a directory structure. It does not care about governance. It allows for rapid prototyping.
- **ops/bin/project-meta:** This is the "smart" tool. It reads docs/ops/registry/PROJECTS.md. It enforces the STELA.md contract.
- **The Benefit:** We can run project-meta in strict CI environments where file creation is banned. We can run project-scaffold in local dev environments without triggering premature governance alarms.

5.2 ops/bin/template: Separating Mutability from Immutability

Current State: The binary handles both generating new agent definitions (Write) and compiling the context manifest (Read).

Refactoring Logic:

- **ops/bin/tpl-factory:** Handles opt/_factory. It is mutation-heavy. It injects "Provenance" blocks (Timestamp, DP ID). It is used by humans during the creation phase.
- **ops/bin/tpl-manifest:** Handles ops/src/manifests. It is read-only. It is deterministic (same input = same output). It is used by machines (CI, Context) to generate CONTEXT.md.
- **The Benefit:** tpl-manifest can be optimized for speed and caching, as it is on the critical path for every context refresh. tpl-factory can focus on user experience and interactivity.

5.3 ops/bin/prune: The Shift to Quartet Logic

Current State: prune is a "Swiss Army Knife" for hygiene, handling archival, tmp scrubbing, and TASK.md restoration.

Refactoring Logic:

1. **Drop reset-task:** The logic to restore TASK.md from ops/src/surfaces/task.md.tpl duplicates logic found in ops/bin/draft (which generates the DP). This capability should be consolidated into draft or template, removing it from prune. prune should only *delete*.
2. **Adopt Quartet Paths:**
   - Change archive target from storage/archives to archives/root.
   - Change scrub target from storage/tmp to var/tmp.
3. **Governance Integration:** Ensure prune respects the immutability of archives/. It should move files *in*, but never delete files *from* the archive root.

5.4 ops/bin/draft vs. tools/lint/dp.sh

Current State: draft generates DPs. tools/lint/dp.sh validates them. Both likely contain logic to parse the TASK.md structure and verify the canonical template hash.[1] **Refactoring Logic:**

- **Centralize Hash Logic:** The canonical template hash should be defined in *one* place (e.g., ops/lib/constants.sh or within tpl-factory). Both draft and lint/dp.sh should reference this single constant.

- **Logic Overlap:** draft implicitly validates the structure it generates. lint/dp.sh explicitly validates it. By ensuring draft uses the *exact same* rendering engine as the validator (tpl-factory), we eliminate the need for draft to have its own complex validation logic. It can simply "trust" the renderer.

6. Strategic Options

Based on the analysis, three distinct strategic paths emerge for executing this transformation. Each offers a different balance of risk, speed, and code reduction.

Option 1: The "Clean Break" (Aggressive Refactor)

Simultaneous implementation of Binary Fission, Quartet Doctrine, and Registry Pattern.

- **Plan:**
  1. **Freeze:** Halt all feature work.
  2. **Topology:** Create var/, logs/, archives/ immediately. Move all artifacts. Update .gitignore.
  3. **Fission:** Split project, template, and verify binaries immediately. Delete the originals.
  4. **Registry:** Implement PATHS.md and refactor *all* CI workflows to use it.
  5. **Refactor:** Rewrite prune and lifecycle scripts to use the new paths and shared libraries.
- **Pros:** Maximum immediate code reduction (>15%). Eliminates technical debt in a single atomic commit. Establishes the new era of governance instantly.
- **Cons:** Extremely high risk of destabilizing the repo. Breaking changes to every workflow simultaneously makes debugging difficult. High "Blast Radius."
- **Risk: Critical.** If the Registry implementation has a bug, all CI gates fail, blocking the fix itself.

Option 2: The "Strangler Fig" (Phased Migration)

Gradual replacement of internals while maintaining interface stability.

- **Plan:**
  1. **Topology:** Create Quartet roots but keep storage/ as a symlink or fallback.
  2. **Shim Binaries:** Create project-scaffold and project-meta, but keep ops/bin/project as a wrapper script calling them.
  3. **Dual-Path Prune:** Update prune to clean both storage/tmp and var/tmp.
  4. **Cleanup:** After one sprint, remove the wrappers and legacy paths.
- **Pros:** Zero disruption to daily operations. "Verify" gates remain green. Low risk.
- **Cons:** Temporary *increase* in code size due to shims and dual-path logic. The 12% reduction is delayed until the final cleanup phase.
- **Risk: Low.**

Option 3: The "Governance-First" Hybrid (Recommended)

Prioritizes the Registry Pattern and Prune Refactor to unlock immediate gains and satisfy the mandate.

- **Plan:**
  1. **Authorize Quartet:** Create archives/, var/, logs/.
  2. **Registry Implementation:** Create docs/ops/registry/PATHS.md and ops/lib/logic/paths.sh.
  3. **Workflow Refactor:** Update CI workflows (pow_policing, etc.) to read from the Registry. This yields immediate, significant code reduction in YAML.
  4. **Prune Cutover:** Refactor ops/bin/prune to strictly use the Quartet paths (no dual support). Remove reset-task logic. This aligns with the "sole focus on storage/archive" mandate.
  5. **Targeted Fission:** Split tools/verify.sh first (high impact, low risk). Split project and template in subsequent DPs.
- **Pros:** Delivers the "Prune" refactor requested. Achieves ~12% reduction through workflow optimization and Prune simplification. Validates the Registry pattern early.
- **Cons:** Leaves project and template overloaded for one more cycle.
- **Risk: Moderate.**

7. Recommendation and Proposal

Proposal: Adopt **Option 3 (The Governance-First Hybrid)**.

This strategy directly addresses the Operator's specific request regarding prune ("prune's focus now should be solely on storage and archive dirs") while satisfying the DP-OPS-0068 mandate. It maximizes the code reduction metric by targeting the most redundant area (CI workflows) via the Registry Pattern, rather than getting bogged down in the complex refactoring of project and template immediately.

Action Plan:

1. **Topology Instantiation:** Create var/tmp/, logs/, and archives/{root,agents,tasks,skills}/. Add .gitkeep files.
2. **Registry Definition:** Create docs/ops/registry/PATHS.md defining "Code Surfaces" (ops/, tools/) and "Canon Surfaces" (PoT.md, TASK.md).
3. **Refactor ops/bin/prune:**
   - Strip reset-task logic.
   - Update archive target to archives/root/.
   - Update scrub target to var/tmp/.
   - Enforce "Results Guard" using the new paths.
4. **Refactor CI Workflows:** Rewrite pow_policing.yml, sop_policing.yml, and task_policing.yml to call a shared script (tools/detect-surface.sh) that reads the Registry, deleting the inline regex logic.
5. **Refactor tools/verify.sh:** Split into verify-tree (checking Quartet roots) and verify-policy.

This path ensures the repository topology is modernized, the governance logic is centralized (SSOT), and the specific overloading of prune is resolved, delivering a robust and leaner system.

8. Conclusion

The *Stela* repository stands at a critical juncture where its governance sophistication (PoT.md) has outpaced its implementation maturity. The overloading of ops/ binaries and the reliance on hardcoded paths in CI workflows represent "Governance Technical Debt" that threatens stability. By executing **Option 3**, the Operator can resolve the immediate overloading of prune, integrate the "Quartet Filing Doctrine," and achieve the mandated 12% code reduction not through brute-force deletion, but through the architectural elegance of the Registry Pattern. This transformation shifts the system from "Imperative Scripting" to "Declarative Governance," ensuring that the repository remains a robust engine of truth.

Works cited

1. dump-full-main-80bb3afd.txt